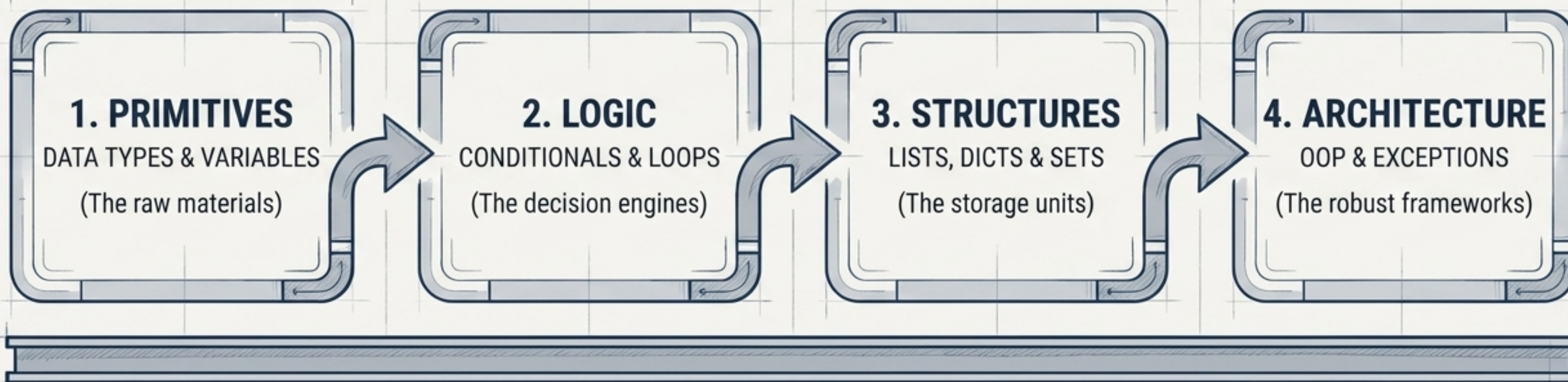
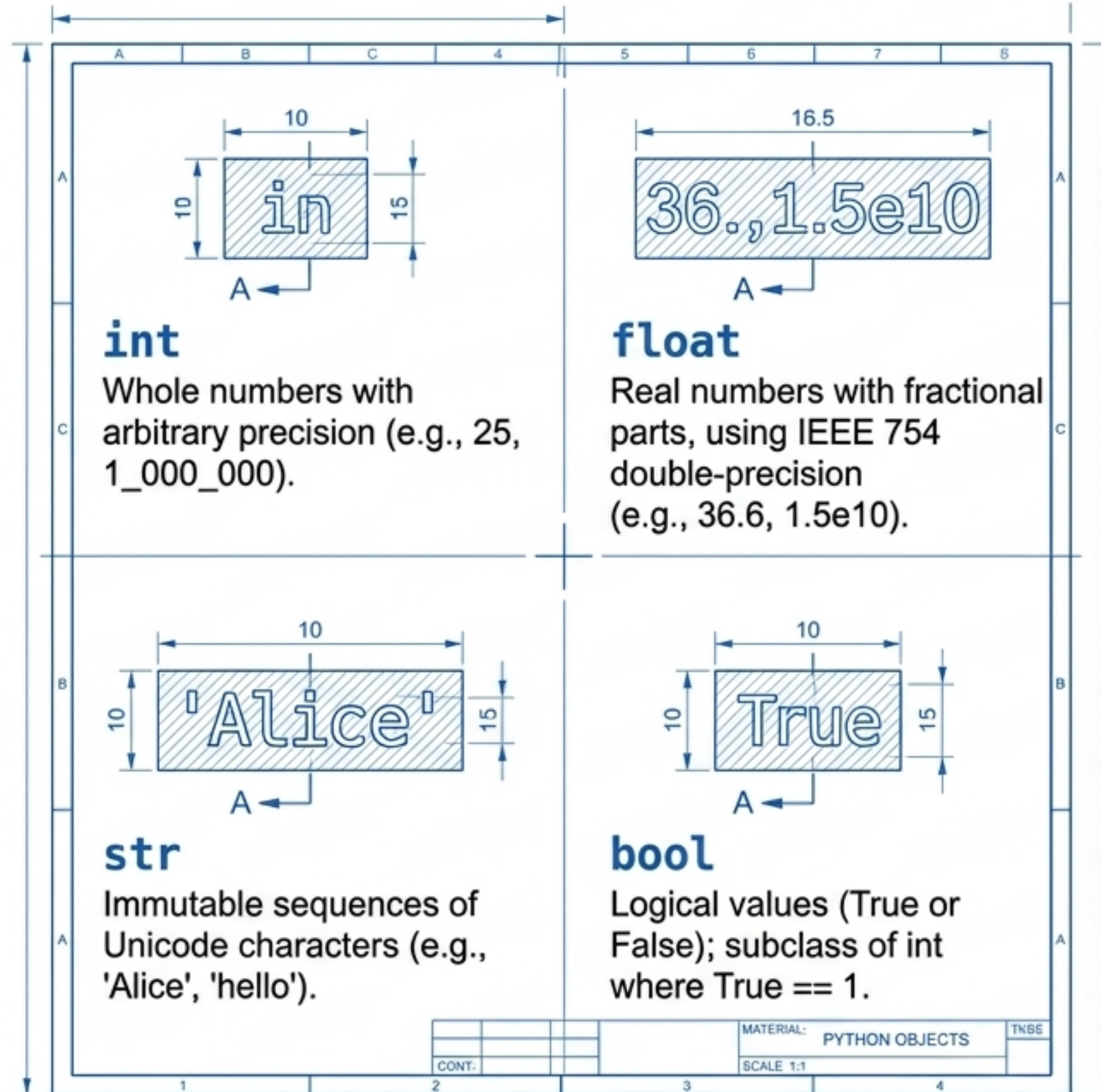


PYTHON FUNDAMENTALS FOR DATA SCIENCE

This module provides the foundational grammar of Python, moving from atomic data types to complex system architectures.



Core Data Types and Dynamic Typing



```
Terminal — python

Dynamic Typing in Python

In statically typed languages, types are declared.
In Python, the interpreter infers the type at runtime.
The built-in type() function inspects this.

temperature = 36.6           # Interpreter infers float
is_active = True             # Interpreter infers bool

print(type(temperature))     # <class 'float'>
print(isinstance(True, int)) # True
```

Variables act as named memory references

CONVENTIONS & RULES



Naming Anatomy: Begin with a letter or underscore. Python is strictly case-sensitive (age vs AGE).



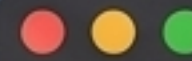
PEP 8 Standard: Use snake_case for variable names (e.g., student_score, discount_pct).



Self-Documenting Code: Meaningful names vastly improve readability in large Data Science projects. x = 85 is uninformative; **student_score = 85** is clear.



Mutability: Variables act as references to memory locations; their values can be overwritten at any time.



Terminal — python

Variable Assignment & f-strings

```
product_name = "Laptop"    # String assignment
price = 75000               # Initial assignment

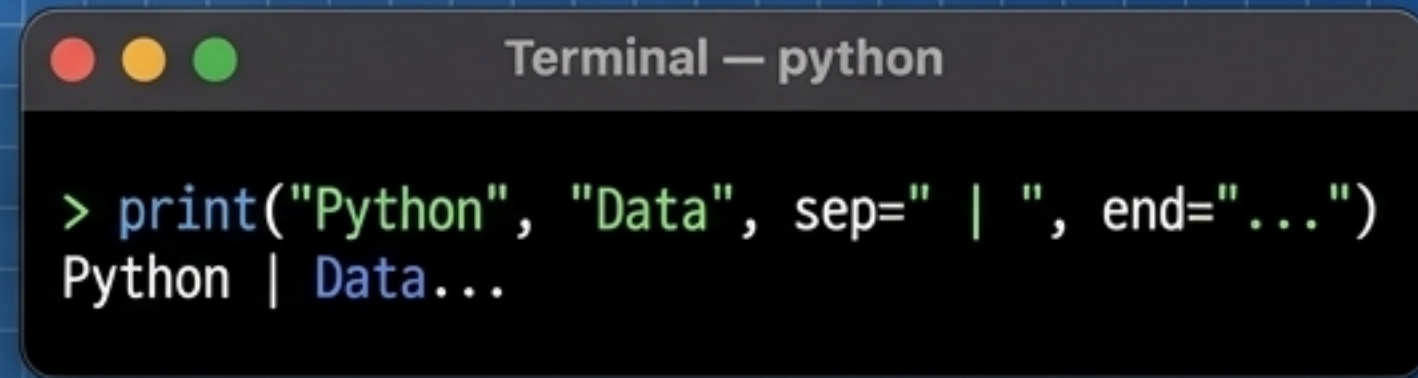
# f-strings evaluate expressions at runtime
print(f"Product: {product_name} costs Rs.{price}")

price = 70000               # Variables are mutable
print(f"Updated Price: Rs.{price}")
```

Interacting with users through Input and Output

Output via `print()`

- Writes values to the standard output console.
- **sep** parameter: Controls the separator between multiple arguments (default is space).
- **end** parameter: Controls what prints at the end (default is a newline `\n`).



```
Terminal — python
> print("Python", "Data", sep=" | ", end="...")
Python | Data...
```

Input via `input()`

- Pauses execution, displays a prompt, and waits for the user to press Enter.

 **Critical Trap:** `input()` always returns a string, regardless of what the user types. Typing 42 returns '42'.



```
Terminal — python
> age = input("Age: ")
Age: 28
> # If user types 28, age stores '28'.
```

Transforming data with Type Conversion

<code>int(x)</code>	Converts strings/floats to integers. Truncates decimals (3.9 -> 3). Raises <code>ValueError</code> on non-numeric strings.
<code>float(x)</code>	Converts integers/strings to floats (5 -> 5.0).
<code>str(x)</code>	Converts any data type to a string representation. Always succeeds.
<code>bool(x)</code>	Converts to True or False based on truthy/falsy rules.

Implicit vs Explicit

Implicit	Explicit
Automatic promotion by Python (e.g., mixed arithmetic: $3 + 1.5 = 4.5$ forces an int to become a float).	Required for converting user input. Always use try-except blocks to validate raw data and avoid fatal <code>ValueError</code> crashes.

Routing logic with Conditional Statements

if: The initial gateway. Evaluates a Boolean condition.

elif (else-if): Mutually exclusive secondary checks. Evaluates sequentially top-to-bottom.

elif (else-if): Mutually exclusive secondary checks. Evaluates.

else: The fallback. Executes only if all preceding conditions are False.

Indentation Rule: Python uses whitespace (strictly 4 spaces), blocks. Incorrect indentation triggers a syntax error.

```
Grade Classifier (ML Label Encoding Logic)

marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 60: # Matches! Block executes.
elif marks >= 40: # Skipped (already matched)
    grade = "C"
elif marks >= 40: # Skipped (already matched)
    grade = "D"
else:
    grade = "F"
```

Building expressions with Comparison and Logical Operators

Comparison		Logical	
==	Equality (tests value equality, not object identity).	and	Evaluates True only if both operands are True.
!=	Inequality.	or	Evaluates True if at least one operand is True.
>, <	Strictly Greater Than / Less Than.	not	Reverses the Boolean state.
>=, <=	Greater Than / Less Than or Equal To.		

Chained Comparisons

Python allows natural mathematical chaining.

Instead of: `(score >= 60) and (score < 80)` → Write: `60 <= score < 80`
(Highly readable and computationally efficient).

PYTHONIC

The hidden logic of Truthy and Falsy values

Any Python object can be evaluated in a Boolean context. This enables concise, elegant checks like 'if my_list:' to verify a list is populated without checking its length.

The Falsy Values

Values that inherently evaluate to False when cast with bool().

⊗ or 0 Numeric Zeros: 0, 0.0

[] () Empty Sequences: "" (String), [] (List), () (Tuple)

{ } Empty Mappings: {} (Dict), set()

☐ or ⊗ The Null Object: None

The Truthy Values

All other values. If an object contains data or represents a non-zero value, it evaluates to True.

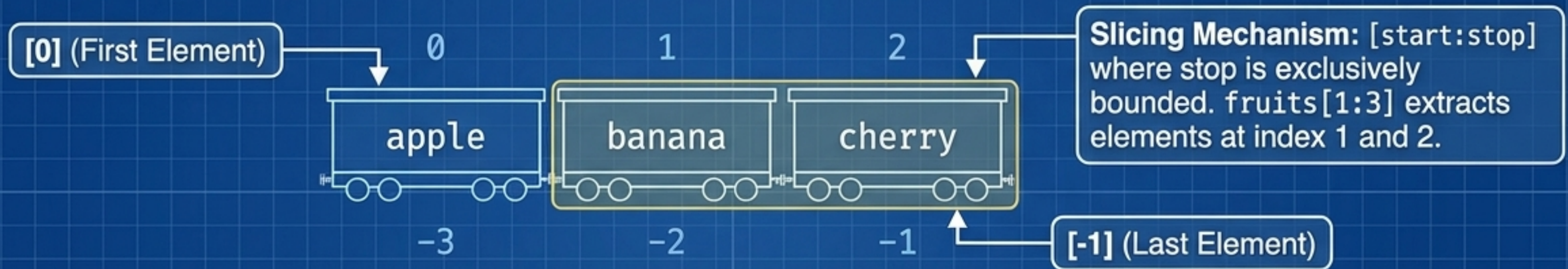


Examples: 42, "hello", [-1, 0, 1], {"key": "value"}.

Storing sequential data in Lists

List Memory Model

Lists are ordered, mutable, and heterogeneous (can hold mixed data types like `[42, 3.14, 'hello']`).



List Mutation Methods

```
.append(x): Inserts element x at the very end of the list.  
.insert(i, x): Inserts element x exactly at index i.  
.remove(x): Scans for and removes the first matching value x.  
del list[i]: Deletes an element precisely by its index.
```

Iteration and automation using Control Flow



The for Loop

Iterates over known sequences (lists, strings, ranges).

- **range(start, stop, step):** Generates an immutable sequence of integers. The stop value is never included.
- **enumerate(iterable):** Wraps a list to yield (index, value) simultaneously.

```
1 | for idx, val in enumerate(['A', 'B']):  
2 |     print(f'{idx}: {val}')
```



The while Loop

Executes continuously as long as a Boolean condition remains True.

Used when the total number of iterations is unknown (e.g., awaiting valid user input).

! Safety Check

Crucial discipline requires manually altering the condition variable inside the loop; failing to do so creates a program-freezing infinite loop.

MATERIAL:
PYTHON CONTROL FLOW LOGIC

SCALE 1:1

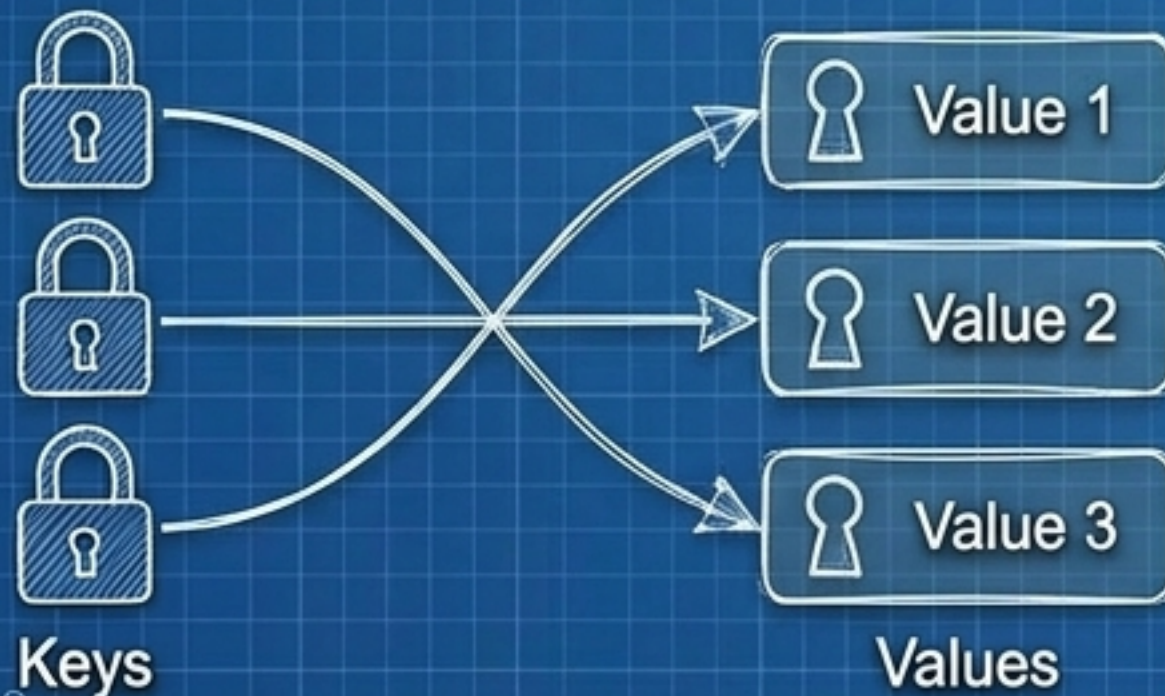
DATE: 2024-05-29

Key-Value mapping with Dictionaries

The Hash Map Architecture

Dictionaries (dict) are insertion-ordered collections of key-value pairs.

- **Performance:** Provide average $O(1)$ lookup, insertion, and deletion speeds.
- **The Key Rule:** Keys must be strictly unique and immutable (strings, integers, tuples).
- **The Value Rule:** Values can be any Python object, including nested lists or other dictionaries.



Extraction Methods

`.get(key)`: Safely extracts a value without crashing if the key is missing (returns `None` instead of `KeyError`).

`.items()`: Returns a view of all (key, value) tuple pairs, perfect for driving a for loop.

```
user = {"name": "Alice", "role": "Data"}  
print(user.get("age")) # Returns None
```

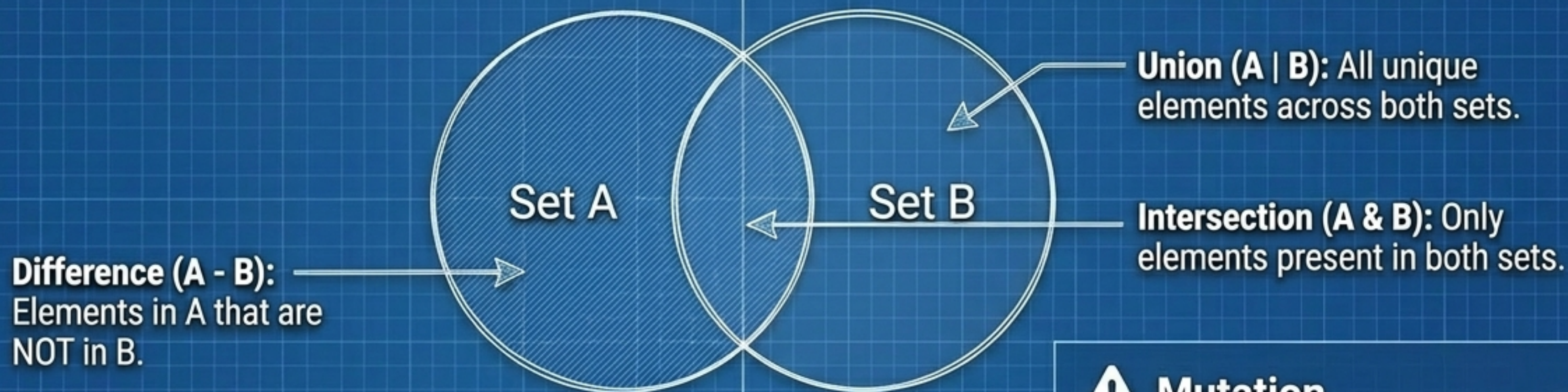
MATERIAL:
PYTHON CONTROL FLOW LOGIC

SCALE 1:1

DATE: 2024-05-29

Deduplication and mathematical logic with Sets

Sets are unordered collections of unique elements. Highly valuable in Data Science for instantly removing duplicates (`unique_ids = list(set(ids))`) or computing vocabulary in NLP.



⚙️ Membership testing (`x in A`):

Highly optimized $O(1)$ complexity, far faster than searching a list.

⚠️ Mutation

Use `A.discard(x)` instead of `remove(x)` to safely delete an item without risking a `KeyError` if the item isn't there.

MATERIAL:
PYTHON CONTROL FLOW LOGIC

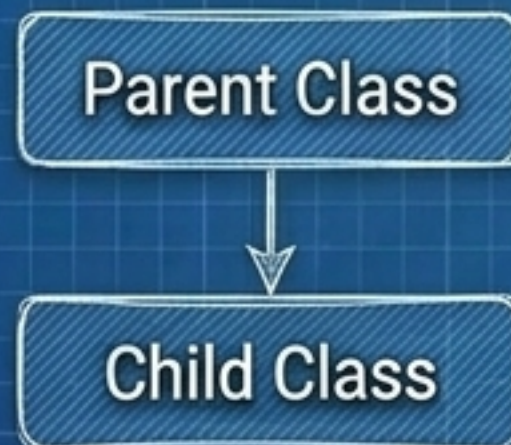
SCALE 1:1

DATE: 2024-05-29

Object-Oriented Programming (OOP)

Architectural Concepts

- **Class vs Object:** A Class is the blueprint defining attributes (data) and methods (behaviour). An Object is the concrete, independent instance.
- **The `__init__` Constructor:** Python's 'dunder' method automatically called upon instantiation to set initial state. 'self' binds attributes to the specific instance.
- **Inheritance & Overriding:** Child classes acquire parent traits (modelling 'is-a' relationships) and can override methods to customize behaviour.



```
Terminal

class Animal:
    def speak(self): return "Sound"

class Dog(Animal): # Dog inherits Animal
    def speak(self): return "Woof!" # Override

bruno = Dog()      # Create Object
```

MATERIAL:
PYTHON CONTROL FLOW LOGIC

SCALE 1:1

DATE: 2024-05-29

Bulletproofing pipelines with Exception Handling

try:

Wraps fragile code that might fail (e.g., file reading, type conversion).



except:

Catches specific exception classes, preventing fatal crashes and allowing graceful fallbacks.



finally:

Always executes. Critical for releasing resources (closing databases/files) regardless of success or failure.

Common Exceptions & raise

- **ValueError**: Right type, inappropriate value (`int('hello')`).
- **TypeError**: Wrong type for operation (`'hello' + 5`).
- **KeyError** / **IndexError**: Missing dictionary key or list index out of bounds.



raise: Manually triggering an error to enforce API rules:

`raise TypeError('Must be integer')`.

MATERIAL:
PYTHON CONTROL FLOW LOGIC

SCALE 1:1

DATE: 2024-05-29

Chapter Summary: From Syntax to Data Science

Python Building Blocks		Data Science Applications
Data Types & Conversion	➡	Safely parsing raw strings from CSVs and APIs into calculable numerics.
Conditionals & Operators	➡	Driving feature engineering logic, thresholding, and data binning.
Lists & Loops	➡	The foundational precursors to vectorised NumPy arrays and batch processing logic.
Dictionaries & Sets	➡	The architectural basis of pandas DataFrames (Dicts) and dataset deduplication (Sets).
OOP & Exceptions	➡	The structural paradigm powering scikit-learn/PyTorch frameworks and production-grade pipeline reliability.

Understanding these fundamental blueprints is the strict prerequisite for mastering Python's advanced ML frameworks